WORKSHOP NATIONAL — BAC+5

PROJECT RESILIENCE-ALPHA

Cyber-résilience et reprise autonome

Dossier d'ingénierie et de gouvernance

Équipe **G<n>** · Session Octobre 2026

[Noms des membres de l'équipe à compléter]

Sommaire

1. Contexte et objectif	7. Résultats des essais
2. Architecture de la solution	8. Éléments de sécurité
3. Rôle des composants	9. Analyse de risques (EBIOS RM)
4. Nature de l'incident et scénarios d'attaque	10. Plan de continuité et de reprise (PCA/PRA)
5. Mécanisme de détection	11. Gouvernance — NIS2 et DORA
6. Mécanisme de remédiation et de reprise	12. Limites assumées et perspectives
	13. Contributions de l'équipe

1. Contexte et objectif

Une plateforme de paiement fictive, exploitée par une entreprise financière, subit des incidents de sécurité qui interrompent ou dégradent ses services. Les équipes d'exploitation parviennent à rétablir le service, mais leurs interventions restent manuelles, longues et peu reproductibles. L'enjeu, pour une activité financière, est la continuité : chaque minute d'indisponibilité retarde les opérations des partenaires et érode la confiance.

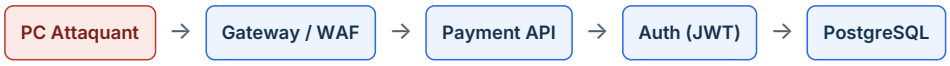
L'objectif de PROJECT RESILIENCE-ALPHA est de démontrer, sur un périmètre maîtrisé, qu'un système d'information de paiement peut **détecter** une anomalie de sécurité, **décider** d'une remédiation et **l'exécuter seul**, puis **vérifier** que le service métier est de nouveau utilisable — sans aucune intervention corrective humaine. Ce n'est pas une plateforme industrielle prête pour la production, mais une chaîne de cyber-résilience crédible, exécutée et défendable.

Toutes les données et opérations sont fictives. Aucune donnée bancaire réelle n'est utilisée.

2. Architecture de la solution

Le démonstrateur est un ensemble de microservices conteneurisés, orchestrés par Docker Compose. Un flux métier nominal traite les transactions ; une chaîne de supervision et de remédiation tourne en parallèle et restaure le service en cas d'incident.

Flux métier nominal



Boucle de reprise autonome



Le point d'entrée public unique est la gateway/WAF ; l'API n'est jamais exposée en direct. La base de données, les secrets, Prometheus et le moteur de remédiation résident sur un réseau interne isolé et ne sont jamais publiés sur le réseau du consortium.

3. Rôle des composants

Composant	Rôle	Technologie
Frontend (dashboard)	Supervision métier : état des services, transactions, incidents, avancement de la reprise	React + Tailwind
Payment API	Initie et traite une transaction fictive ; expose `/metrics` et `/health`	FastAPI
Auth	Authentifie et émet des jetons JWT	FastAPI, bcrypt
PostgreSQL	Données métier fictives (comptes, transactions)	Postgres 16
Gateway / WAF	Unique entrée publique de l'API ; filtre les requêtes malveillantes	nginx + ModSecurity (OWASP CRS)
Observabilité	Métriques et journaux centralisés, lisibles par le jury	Prometheus, Loki, Promtail, Grafana
Détecteur ML	Repère un comportement anormal à partir des métriques	Isolation Forest (scikit-learn)
Moteur de remédiation	Reçoit l'incident, décide, agit sur les conteneurs, vérifie le retour à la normale	Python + proxy socket Docker
Assistant RAG	Aide l'opérateur à consulter les procédures (lecture seule)	Ollama (≤8B) + LangChain + Chroma
SOPS	Chiffre les secrets du dépôt	SOPS

4. Nature de l'incident et scénarios d'attaque

Le projet distingue deux menaces de nature différente, qui appellent deux réponses différentes. Ne pas les confondre est un point central du sujet.

Attaque principale — flood (épuisement de ressources)

Depuis le PC Attaquant, un grand nombre de requêtes concurrentes est envoyé à la Payment API. C'est une atteinte à la **disponibilité** : la latence explose et des erreurs apparaissent. L'effet est mesurable par la chaîne de supervision, donc détectable par le modèle, et la réponse — restaurer la disponibilité — est automatisable proprement.

Attaque secondaire — injection SQL

Une requête contenant une charge d'injection (`1' OR '1'='1`) est envoyée à l'API. C'est une tentative de **compromission**. La réponse attendue n'est pas un redémarrage, mais un blocage : le WAF reconnaît le motif, renvoie une erreur 403 avant même l'authentification, et journalise l'événement pour investigation.

> Distinguer disponibilité et compromission : on restaure un service tombé, on bloque et on trace une tentative d'intrusion. Les deux réponses ne sont pas interchangeables.

5. Mécanisme de détection

La détection repose sur un modèle **Isolation Forest**, un algorithme de machine learning non supervisé. Aucun seuil n'est codé en dur, conformément à l'exigence du sujet.

Au démarrage, le détecteur observe le service en fonctionnement normal pendant environ 60 secondes et collecte des vecteurs de caractéristiques : débit de requêtes, ratio d'erreurs et latence p95, tirés de Prometheus. Il s'entraîne sur ces observations, apprenant la forme du trafic « normal ». Ensuite, il score chaque nouvelle mesure et isole les points atypiques.

Pour éviter les faux positifs, le détecteur exige **trois anomalies consécutives** avant de déclencher, puis s'impose une pause après action. Il ne corrige rien lui-même : détection et action sont volontairement séparées. Lorsqu'une anomalie est confirmée, il envoie un incident au moteur de remédiation.

6. Mécanisme de remédiation et de reprise

Le moteur de remédiation reçoit l'incident, décide d'une action, l'exécute puis **vérifie** le résultat. Pour une perte de disponibilité, l'action est le redémarrage ciblé du service touché. La vérification est décisive : le moteur exécute une transaction de contrôle, et le service n'est considéré comme rétabli que s'il **répond réellement**. Le `/health` de l'API effectue un vrai `SELECT` en base : un conteneur « up » mais dont la base ne répond pas reste « unhealthy ». Un simple redémarrage ne vaut donc pas preuve de reprise.

Si la vérification échoue, l'incident passe en statut `escalate` plutôt que `resolved`, et l'échec est tracé pour qu'un humain prenne le relais.

> Choix défendable : nous n'avons pas mis de politique `restart` automatique de Docker sur la Payment API. Sinon Docker la relancerait en une seconde et masquerait l'incident. Nous voulons que ce soit la chaîne détection + remédiation qui restaure le service — c'est cela, la reprise autonome démontrée.

7. Résultats des essais

Les essais ont été menés sur le démonstrateur réel. La boucle complète a été exécutée via le script `scripts/demo.sh` (avant → attaque → détection → remédiation → après).

7.1 Reprise autonome (run de référence)

0,016 s	1,27 s	0,019 s	9,8 s
transaction AVANT	transaction PENDANT	transaction APRÈS	RTO mesuré

Phase	Observation
Avant	Transaction de contrôle : 200 OK en 0,016 s (service sain)
Attaque	Flood : 9 879 requêtes, dont 6 688 erreurs / timeouts (saturation réelle)
Pendant	Transaction dégradée : 200 en 1,27 s (×80 plus lent)
Détection + remédiation	Incident détecté et résolu automatiquement : `INC-0a39d3ad`, statut `resolved`
Après	Transaction de contrôle : 200 OK en 0,019 s (service rétabli)

Le temps de reprise (RTO) observé varie de 10 à 35 secondes selon le moment où l'attaque tombe dans le cycle d'échantillonnage du détecteur, et reste toujours sous la cible de 60 secondes.

7.2 Blocage de l'injection SQL

Le script `attack/sqli.sh` envoie une charge d'injection. Le WAF renvoie **HTTP 403** : la requête est bloquée avant l'authentification et journalisée. Résultat conforme à l'attendu.

7.3 Assistant RAG

L'assistant répond aux questions des opérateurs en s'appuyant sur les documents du projet (champ `sources` non nul), avec un modèle de langage local (≤8B). Embeddings par `nomic-embed-text`, génération par `llama3.1:8b`. Aucune donnée ne sort du système.

8. Éléments de sécurité

Mesure	Mise en œuvre
Moindre privilège du moteur de remédiation	Accès Docker via un proxy n'autorisant que lister et redémarrer — ni création d'image, ni volumes, ni exec
Cloisonnement réseau	Base, secrets, Prometheus, remédiation sur un réseau `internal` jamais publié ; seuls dashboard, API (via WAF) et Grafana exposés
Filtrage applicatif	WAF ModSecurity + règles OWASP CRS en amont de l'API
Gestion des secrets	Chiffrement SOPS ; aucun secret en clair dans l'archive ; `.env` hors dépôt
Authentification	Mots de passe hachés (bcrypt), jetons JWT signés
Traçabilité	Journaux centralisés (Loki) ; événements WAF exploitables pour investigation

Piste de durcissement identifiée : passer les JWT en RS256 (clé privée côté Auth, clé publique seule côté API) pour que l'API ne puisse jamais forger de jeton.

9. Analyse de risques (EBIOS RM)

Analyse structurée selon la méthode EBIOS Risk Manager. Chaque mesure de traitement est rattachée à un composant réel du démonstrateur.

Scénario	Vrais. (1-4)	Gravité (1-4)	Niveau	Mesure
Flood → indisponibilité API	4	3	Élevé	Rate-limit + remédiation auto
Injection SQL via l'API	3	4	Élevé	WAF + requêtes paramétrées
Remédiation détournée (droits Docker)	2	4	Élevé	Proxy socket en moindre privilège
Secret exposé dans le dépôt	3	3	Moyen	SOPS + .gitignore
Faux positif ML → action inutile	3	2	Moyen	Seuil de 3 anomalies + cooldown

10. Plan de continuité et de reprise (PCA/PRA)

Objectifs

RTO visé : moins de 60 secondes, mesuré en direct et affiché au Clip Chrono. RPO : zéro sur les transactions validées, persistées en base avant acquittement.

Scénarios et réponses

Scénario	Type	Réponse	Automatique ?
Flood → indisponibilité	Disponibilité	Redémarrage ciblé + vérification	Oui
Injection SQL	Compromission	Blocage WAF + trace SIEM	Oui (blocage)
Corruption de données	Intégrité	Restauration depuis sauvegarde	Non (hors périmètre)

Risques résiduels après reprise

Le redémarrage restaure la disponibilité mais n'élimine pas la cause : l'attaquant peut réitérer. Un faux négatif du modèle retarderait la détection. Il n'y a pas de bascule multi-site : une panne matérielle du PC Hôte entraîne une interruption.

11. Gouvernance — NIS2 et DORA

Pour une activité financière fictive, nous distinguons les exigences illustrées par le prototype de celles qui relèvent de l'organisation et ne peuvent être démontrées en quatre jours.

Exigence	Cadre	Démontré ?
Détection et réponse aux incidents	NIS2 / DORA	Oui — chaîne détection → remédiation → journalisation
Tests de résilience opérationnelle	DORA	Oui — attaque + preuve de reprise
Objectifs de continuité (RTO/RPO)	DORA	Oui — RTO < 60 s mesuré, RPO 0
Traçabilité et journalisation	NIS2	Oui — Loki + événements WAF
Notification aux autorités (72 h)	NIS2	Non — processus organisationnel
Gestion du risque tiers (TIC)	DORA	Non — hors prototype

Recommandation à une direction : le prototype prouve la capacité technique de détection et de reprise. La priorité d'industrialisation serait la journalisation et la notification d'incident, premier point contrôlé en cas d'audit.

12. Limites assumées et perspectives

Le projet est un prototype de quatre jours, pas une plateforme de paiement industrielle. Le modèle apprend sur un baseline court, sensible au bruit. La remédiation est un répertoire d'actions prédéfinies, pas une IA décisionnelle ; chaque action est toutefois vérifiée et tracée, et l'architecture permet d'en ajouter (isolation réseau, rate-limit, scaling) sans tout refaire. La reprise couvre la disponibilité ; une compromission avérée relève de l'investigation, pas du redémarrage. Il n'y a pas de redondance multi-site.

13. Contributions de l'équipe

[À compléter par l'équipe — une contribution identifiable par personne, conformément au sujet]

Membre	Pilier(s) principal(aux)	Contribution identifiable
	Développement (API, Auth, DB, frontend)	
	Infrastructure (Compose, Ansible, observabilité)	
	IA / Data (ML + RAG)	
	Cybersécurité (WAF, attaque, secrets)	
	Gouvernance (EBIOS, PCA/PRA, NIS2/DORA)	

Détails techniques volumineux : voir le dépôt de code livré (`Workshop2026-M2-G<n>-Code.zip`), qui contient les sources, les configurations, la documentation d'API et le README de déploiement.